

→JS-4

Loops

Used to iterate a piece of code

→For loops

for loop

```
for(initialisation; condition; updation) {  
    //do something  
}
```

```
for (let i=1; i<=5; i++) {  
    console.log(i);  
}
```

→Print all Odd numbers 1 to 15

jcreddy2096@gmail.com

$i++ \rightarrow +1$
 $i-- \rightarrow -1$

Print all odd numbers (1 to 15)

start → 1, 3, 5, 7, 9, 11, 13, 15 → end

1 +2
3 +2
5 +2
7 +2
9 +2
11
13
15 update

```
for (let i=1 ; i<=15 ; i=i+2) {  
    console.log(i);  
}
```

start end update



→ Print even numbers

Print all even numbers (2 to 10)

2 ← start
4
6
8
10 ← end

```
for (let i = 2; i <= 10; i = i + 2) {  
  console.log(i);  
}
```

Handwritten annotations: "start" under i=2, "end" under i<=10, "update" under i=i+2.

→ Infinite loops

Infinite Loops

```
for(let i=1; i>=0; i++) {  
  }  
  
for(let i=1; i<=5; i--) {  
  }  
  
for(let i=1; ; i++) {  
  }
```

Handwritten note: "ending condition" with an arrow pointing down to "missing true".

Print the multiplication table for 5

start → 5
10
15
20
25
30
35
40
45
end → 50

+5
+5
+5

```
for (let i = 5; i <= 50; i = i + 5) {  
  console.log(i);  
}
```

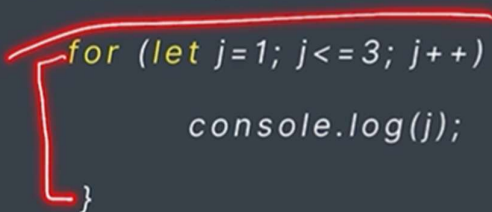
Handwritten annotations: "start" with an arrow pointing to 5, "end" with an arrow pointing to 50, and a bracket grouping the numbers 5 to 50.

→ print multiplication of 5

→ nested for loop

Nested for loop

```
for (let i=1; i<=3; i++) {  
  for (let j=1; j<=3; j++) {  
    console.log(j);  
  }  
}
```



→ While loop

while loop

```
while (condition) {  
  //do something  
}
```

```
let i = 1;  
while (i <= 5) {  
  console.log(i);  
  i++;  
}
```

while loop

```
while (condition) {  
  //do something  
}
```

ddy2096@gmail.com

```
let i = 1;
```

```
while (i <= 5) {
```

```
  console.log(i);
```

```
  i++;
```

✓

i=1 (T)

i=2 (T)

i=3 (T)

i=4 (T)

i=5 (T)

i=6 (F)

1
2
3
4
5

→ Favorite movie

Favorite Movie

```
let favMovie = "Avatar";
```

```
let guess = prompt("");
```

```
while ((guess != favMovie) && (guess != 'quit')) {
```

```
  console.log("wrong");
```

```
  guess = prompt("");
```

```
}
```

2 condition

quit

1) right guess $\Rightarrow$ guess = favMovie
2) guess = "quit"

→ break keyword

break keyword

loop execution stop

→ Loops with Arrays

Loops with Arrays

```
let fruits = ["mango", "apple", "banana", "litchi", "orange"];

for(let i=0; i<fruits.length; i++) {
  console.log(i, fruits[i]);
}
```

i=0
i=1
i=2
i=3
i=4

→ Nested loop Arrays

Loops with Arrays

Nested Loops with Nested Arrays

```
let heroes = [ ["ironman", "spiderman", "thor"], ["superman", "wonder woman", "flash"]];

for(let i=0; i<heroes.length; i++) {
  console.log(`List #${i}`);
  for(let j=0; j<heroes[i].length; j++) {
    console.log(heroes[i][j]);
  }
}
```

Handwritten annotations: A bracket above the first array ["ironman", "spiderman", "thor"] is labeled "marvel". A bracket above the second array ["superman", "wonder woman", "flash"] is labeled "dc". A blue circle with a checkmark is placed under the first array. The number "1" is written below the second array.

→ For of loop

for of loop

for > counter
while i

```
for (element of collection) {  
  //do something  
}
```

array
string

```
let fruits = ["mango", "apple", "banana", "litchi", "orange"]; jcreddy
```

```
for(fruit of fruits) {  
  console.log(fruit);  
}
```

```
for(char of "apnacollege") {  
  console.log(char);  
}
```

→ for of loops in nested loop

Nested for of loop

```
let heroes = [ ["ironman", "spiderman", "thor"], ["superman", "wonder woman", "flash"]];
```

```
for(list of heroes) {  
  for(hero of list) {  
    console.log(hero);  
  }  
}
```

jcreddy2096@gmail

→JS-5

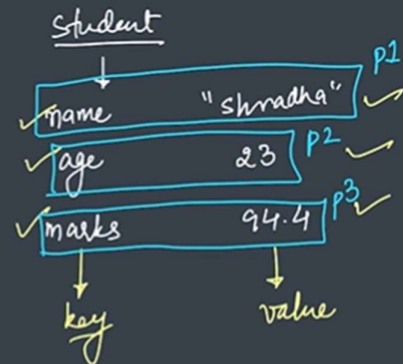
→Object Literals

JS Objects Literals

Used to store keyed collections & complex entities.

property => (key, value) pair

Objects are a collection of properties



```
js / ...
let student = {
  name: "shradha",
  age: 23,
  marks: 94.4
};

let student2 = ["shradha", 23, 94.4];
```

→we get the clarity in object literals

than arrays.

```
const student = {
  name: "shradha",
  age: 23,
  marks: 94.4
};
```

JS Objects Literals

```
let delhi = {
  latitude: "28.7041° N",
  longitude: "77.1025° E"
};
```

```
const student = {
  name: "shradha",
  age: 23,
  marks: 94.4,
  city: "Delhi"
};
```

```
const student = {
  name: "shradha",
  age: 23,
  marks: 94.4
};

const item = {
  price: 100.99,
  discount: 50,
  colors: ["red", "pink"]
};
```

→ Thread/Twitter post

Thread / Twitter Post

Create an object literal for the properties of thread/ twitter post which includes

- username
- content
- likes
- reposts
- tags

post

→ Get values

Get Values

```
let student = {  
  name : "shradha",  
  marks : 94.4  
};
```

```
student["name"]
```

```
student.name
```



```

< ▶ {username: '@shradhakhapra', content: 'This is my #firstPost', likes: 150, repost
s: 5, tags: Array(2)}
> post["content"];
< 'This is my #firstPost'
> post["likes"];
< 150
> post.content;
< 'This is my #firstPost'
> post."content";
? Uncaught SyntaxError: Unexpected string VM315:1
> post[content];
? ▶ Uncaught ReferenceError: content is not defined VM326:1
  at <anonymous>:1:6
> post["likes"];
< 150
> post.likes
< 150
> post.reposts
< 5
> post.username
< '@shradhakhapra'
> post.tags
< ▶ (2) ['@apnacollege', '@delta']
> post.tags[0]
< '@apnacollege'
>

```

→ Get values

Get Values

JS automatically converts objects keys to strings.

Even if we made the number as a key, the number will be converted to string.

```

true: "c"
undefined: "e"
  > [[Prototype]]: Object
> "null"
< 'null'
> null
< null
> obj[1]
< 'a'
> obj[2]
< 'b'
> obj[null]
< 'd'
> obj[true]
< 'c'
> obj[undefined]
< 'e'
> obj.1
Uncaught SyntaxError: Unexpected token
> obj.null
< 'd'
> obj.undefined
< 'e'
> obj.true

```

→ App/update value

Add/ Update Value

- Change the city to "Mumbai"
- Add a new property, gender: "Female"
- Change the marks to "A"

```
const student = {
  name: "shradha",
  age: 23,
  marks: 94.4,
  city: "Delhi"
};
```

```

student.gender = "female";
'female'
student.gender
'female'
student
  > {name: 'shradha', age: 23, marks: 94.4, city: 'Mumbai', gender: 'female'}
student.marks
94.4
student.marks = "A";
'A'
student.marks
'A'
student
  > {name: 'shradha', age: 23, marks: 'A', city: 'Mumbai', gender: 'female'}
student.marks = [99, 89, 74];
(3) [99, 89, 74]
student
  > {name: 'shradha', age: 23, marks: Array(3), city: 'Mumbai', gender: 'female'}
    age: 23
    city: "Mumbai"
    gender: "female"
    marks: (3) [99, 89, 74]
    name: "shradha"
  > [[Prototype]]: Object

```

```
> student
< ▶ {name: 'shradha', age: 23, marks: 94.4, city: 'Delhi'}
> delete student.marks;
< true
> student
< ▶ {name: 'shradha', age: 23, city: 'Delhi'}
> delete student.city;
< true
> student
< ▶ {name: 'shradha', age: 23}
> |
```

→ Object of objects

Object of Objects

Storing information of multiple students

```
const classInfo = {
  aman : {
    grade: "A+",
    city: "Delhi"
  },
  shradha : {
    grade: "A",
    city: "Pune"
  },
  karan : {
    grade: "O",
    city: "Mumbai"
  }
};
```

Handwritten annotations: Blue brackets on the left group the student objects under the label 'obj'. A yellow bracket on the right groups the entire object under the label 'obj'.

```

> student
< {name: 'jai', age: 19, marks: 96, money: '1Lakh'}
> classInfo
< {vinayaka: {grade: 'A+', city: 'kanipakam'}, jai: {grade: 'A', city: 'bkpalli'}, siva: {grade: 'A', city: 'bkpalli'}}
> classInfo.vinayaka
< {grade: 'A+', city: 'kanipakam'}
> classInfo.siva.city
< 'bkpalli'
> classInfo.siva.city="rajyasabha"
< 'rajyasabha'
> classInfo
< {vinayaka: {grade: 'A+', city: 'kanipakam'}, jai: {grade: 'A', city: 'bkpalli'}, siva: {grade: 'A', city: 'rajyasabha'}}
>

```

→ Array of Objects

Array of Objects

Storing information of multiple students

```

const classInfo = [
  {
    name: "aman",
    grade: "A+",
    city: "Delhi"
  },
  {
    name: "shradha",
    grade: "A+",
    city: "Pune"
  },
  {
    name: "karan",
    grade: "O",
    city: "Mumbai"
  }
];

```

```

classInfo
  (3) [{-}, {-}, {-}] ⓘ
    0: {name: 'aman', grade: 'A+', city: 'Delhi'}
    1: {name: 'shradha', grade: 'A', city: 'Pune'}
    2: {name: 'karan', grade: 'O', city: 'Mumbai'}
    length: 3
    [[Prototype]]: Array(0)

classInfo[0]
  {name: 'aman', grade: 'A+', city: 'Delhi'}

classInfo[1]
  {name: 'shradha', grade: 'A', city: 'Pune'}

classInfo[2]
  {name: 'karan', grade: 'O', city: 'Mumbai'}

classInfo[1].name
  'shradha'

classInfo[1].grade
  'A'

classInfo[1].city
  'Pune'

classInfo[1].city = "Gurgaon";
  'Gurgaon'

classInfo[1]
  {name: 'shradha', grade: 'A', city: 'Gurgaon'}

```

```

classInfo[1]
  {name: 'shradha', grade: 'A', city: 'Gurgaon'}

classInfo[1].gender = "female";
  'female'

classInfo[1]
  {name: 'shradha', grade: 'A', city: 'Gurgaon', gender: 'female'}

```

→ Math objects

Math Object

Properties

Math.PI

Math.E

Methods

Math.abs(n)

Math.pow(a, b)

Math.floor(n)

Math.ceil(n)

Math.random()

```

> Math
< Math {abs: f, acos: f, acosh: f, asin: f, asinh: f, ...}
> Math.PI
< 3.141592653589793
> Math.E
< 2.718281828459045
> Math.abs(12);
< 12
> Math.abs(-12);
< 12
> Math.abs(-12.5);
< 12.5
> Math.pow(2, 4);
< 16
> Math.pow(2, 5);
< 32

```

→ Math.floor: it will give the numbers which is smaller and equal to the value :<= for floor

```

> Math.floor(5)
< 5
> Math.floor(5.5)
< 5
> Math.floor(5.999999999)
< 5
> Math.floor(-5)
< -5
> Math.floor(-5.5)
< -6
>

```

→ Math.ceil : it gives the number greater than the given number

```

< -6
> Math.ceil(5)
< 5
> Math.ceil(5.5)
< 6
> Math.ceil(5.000000001)
< 6
> Math.ceil(-5)
< -5
> Math.ceil(-5.5)
< -5

```

→ Math.random: it will give the value between the 0 to 1.

```

Math.random()
0.10191202663404164
Math.random()
0.13987166722029798
Math.random()
0.8144614591017076
Math.random()
0.7534498216923924
Math.random()
0.7413426254246709
Math.random()
0.6190270051920681

```

→ Random Integers

Math.random() → 0 to 1

Random Integers

From 1 to 10

Step1: `let num = Math.random();` 0.46747741318127045

Step2: `num = num * 10;` 4.674774131812704

Step3: `num = Math.floor(num);` 4

Step4: `num = num + 1;` 5

Random Integers

From 1 to 10

```

let random = Math.floor(Math.random() * 10) + 1;
undefined
random
4

```

→ Practice Qs

Qs

Generate a random number between 1 and 100.

$$\text{Math.floor}(\text{Math.random()} * 100) + 1$$

0 to 99

max

Generate a random number between 1 and 5.

21 to 25

21, 22, 23, 24, 25